

Week 3 Design Report: Incremental Urban Data Platform

ID2221 Data-Intensive Computing | September 2026

1. Requirements and release design

In Weeks 1 and 2, we built a batch pipeline that joins taxi trips with weather, air quality and zone references. Rebuilding every table for each daily delivery would repeat much of that work. For Week 3, we kept the existing data model and added a path for new records and corrections. Our main concern was consistency: a corrected weather observation must reach the relevant trips and summaries, while unrelated results should remain unchanged.

The platform has three processing layers. Bronze contains validated source records, Gold contains enriched taxi trips, and four product tables contain analytical summaries. Delta commits are atomic within one table. We therefore publish a manifest of table versions only after all required stages succeed, so readers can select a consistent completed state.

The update generator is deterministic with seed 2221 and streams the source files rather than loading them into memory. Taxi output preserves the exact Parquet schema. It samples 7.5% of the raw source count as realistic new rows, moves pickup and drop-off times to the seven days following the source maximum, and copies another 1.5% unchanged as duplicate records. Weather and air quality each add 168 consecutive hourly observations after that source's own maximum. Weather adds numeric humidity; air quality adds numeric aqi. A few valid February-March observations and three zone labels are revised so propagation of corrections can be tested even though the three source clocks do not end in the same week. File counts, schemas, time ranges, byte sizes and SHA-256 checksums are stored in `data/updates/manifest.json`.

The release contains valid records. We put missing keys, invalid values and unsupported columns in separate test fixtures, which lets us evaluate error handling without mixing artificial errors into the performance workload.

2. Validation, schema evolution and Bronze MERGE

Validation is driven by `config/datasets.yaml`. Each dataset declares its raw columns, primary key, required standardized fields, timestamp fields, numeric ranges, foreign keys, partitioning and allowlisted additive columns. Generic rules detect missing and incomplete keys, failed parses, timestamps outside 2000-2100, impossible values, batch duplicates and reference IDs absent from the zone table. Dataset transforms handle only source-specific naming, casting and timestamp normalization. A new dataset normally requires a configuration block and, only when its raw shape differs, one transform function.

The strict Week 3 baseline re-reads all historical inputs. A valid row wins over an invalid row with the same key; remaining ties use deterministic serialized payload order. Rejected rows are written with concrete reason strings to per-dataset Delta quarantine tables. This intentionally changes historical counts from Week 2 where stronger rules identify additional defects. The design does not rewrite earlier benchmark evidence; the evaluation report states the new baseline and its rejection reasons separately.

Raw schemas from the accepted baseline are saved as manifests. An update must contain all expected columns. Any unknown field or changed raw type fails that dataset. Only humidity: string and aqi: string are allowlisted additions; their transforms produce `humidity_pct_v2` and `aqi`, and their declared ranges are validated. A failure is recorded and does not prevent other datasets from being inspected, but it blocks publication of an incomplete analytical snapshot. Additive evolution is enabled only around the relevant MERGE and is reset immediately afterwards.

Bronze uses Delta MERGE by stable keys. Zone ID, weather observation hour, and air-quality station/parameter/POC/hour support insert, exact duplicate and same-key correction classification.

Taxi has no upstream trip identifier. Its trip_id is a SHA-256 hash of all named raw fields with null positions preserved, so it reliably detects an exact duplicate; a changed taxi payload is a distinct trip. Reliable correction handling would require an upstream trip identifier.

Each Bronze table has Change Data Feed (CDF) enabled. The orchestrator combines current-batch keys with all inserts and post-images after the last published Bronze versions. This includes pending changes from a failed run even when the next batch has new records. Product recovery also starts at the published Gold version, rather than the beginning of the latest attempt. Readers keep using the old publication until all required work succeeds.

3. Partial Gold integration and analytical products

Source change keys are mapped to affected taxi trips. New taxi keys map directly. A changed weather hour maps to trips in that UTC pickup hour. A zone change maps to pickup or drop-off location IDs. An NYC air-quality key maps its state/county code to borough and joins trips by borough-hour. Environmental updates with no matching trip return an empty affected set and produce no Gold or product commit.

Only affected trips are re-enriched and merged into integrated Gold. The joins retain the Week 2 UTC storage model and New York reporting timezone. Original relative_humidity_pct remains for compatibility; humidity_pct_v2 carries the new measurement. PM2.5 aggregation still averages instruments within a station and then gives stations equal borough-hour weight; aqi_max adds the new AQI context. Gold CDF records pre-images, post-images and inserts for product selection. We pair each row's before and after values within its commit, and keep all intermediate aggregation groups. If a correction is later reversed, a product partly written before failure may still contain the intermediate group; it must be recomputed or removed even when the final source value matches the old one.

The four products use different incremental grains. Daily mobility recomputes changed local-date and borough groups. Taxi-zone statistics recomputes changed local-month and pickup-zone groups. Air-quality impact replaces changed borough-hours. Weather impact replaces changed zone and weather-code groups. When new taxi days extend the live analysis window, the weather product is rebuilt because its explicit hour spine changes every zone's observed-hour denominator. Otherwise it refreshes affected groups, including zero-trip hours when zone labels or citywide weather categories change. The product registry records source versions, schema, refresh times, row counts, storage size and parameters.

The six Week 2 queries continue to run against the integrated schema. The live window ends at the day after the latest accepted New York pickup date, while historical_end_date_exclusive retains 1 April 2024 for Week 2 comparison. Monthly demand change is NULL when either month is incomplete, avoiding a misleading comparison for partial April.

4. Publication and monitoring architecture

Delta commits are atomic within one table. We publish a manifest only after Bronze, Gold and all products finish. It records their exact Delta versions and the analysis window. Readers use these versions throughout a query and retain the previous manifest if an update fails.

Every dataset and stage appends a record to data/monitoring/pipeline_runs with run ID, UTC timestamps, duration, processed, inserted, updated, duplicate and rejected counts, schema version, validation-failure count, Delta versions and an error field. A second Delta table stores successful publications. Four Spark SQL reports answer which dataset fails validation most frequently, which takes the longest, how many rows each run rejected, and how duration changes over runs. These metrics connect a failure to its source, stage and version and show whether growth or skew changes operating cost.

Production monitoring would additionally collect input freshness, executor failures, shuffle spill, skew, file counts, source-to-publication latency, quarantine age and service-level alerts. The local implementation keeps the required metrics durable and measures the time spent writing them, so monitoring cost is visible rather than assumed negligible.

5. Engineering decisions, trade-offs and scaling

The configuration-driven loader from Week 1 was useful because we could add required fields and reference checks without creating four separate pipelines. Schema checks, quarantine and MERGE classification now share the same dataset definitions. Week 2's explicit aggregation groups and stored sums and counts also made selective refresh easier. The largest changes were coordinating publication across tables and tracing environmental corrections back to trips.

For the existing rule families, a new constraint only needs a configuration entry, such as a numeric range or foreign-key reference. Dataset-specific transformations still need code, and a new kind of rule, such as a relationship between two measurements, needs an extension to the validator. We do not claim that arbitrary rules or future datasets can be supported through configuration alone. A rule registry would make that extension cleaner. Missing-field and duplicate checks are generic; numerical thresholds, taxi-zone references and trip-duration checks depend on the dataset. Column renames, removals and type changes require a planned migration and updates to downstream queries.

Strict validation performs additional scans and primary-key shuffles. CDF and Delta logs add storage. Per-grain MERGE avoids broad recomputation but is more complex than overwrite and can create small files over many releases. Periodic compaction and retention policies would be needed in a long-running deployment. The chosen product tables are small and unpartitioned; partitioning them now would increase file overhead. Bronze taxi and air quality retain date partitions, while updates use partition values derived from normalized UTC timestamps.

For ten cities, every key and analytical grain should include `city_id`. Timezone, zone references, weather provenance and validation ranges should move to per-city configuration. The platform should use an orchestrator with durable run state, object-store checkpoints and a catalog rather than local paths. Incremental joins should prune city/date partitions, detect skew and compact small files. Broadcast decisions should use observed table size, because a lookup that is small for NYC may not remain small across cities. Publication manifests and source offsets should be replicated and monitored, and CDF retention must exceed the longest supported recovery interval.

We tested recovery by interrupting the pipeline after Gold committed and after a product committed. The publication must remain unchanged on failure; replay must then agree with products recomputed from current Gold. A further case adds new keys while earlier changes are still pending. These cases motivated the publication-based CDF boundary and commit-specific pairing described above. The runner assumes a single writer and retained CDF history. A shared deployment would additionally need writer coordination and a durable scheduler.

If we redesigned the platform, we would measure correction size before choosing between partial and full refresh. Updating a zone label can affect millions of trips, so an incremental MERGE is not always cheaper than rebuilding Gold. Finally, we would add source trip identifiers where available and verify the weather timezone. The current UTC assumption can shift weather associations by four or five hours and limits the interpretation of weather results.

References

- Delta Lake MERGE and schema evolution: <https://docs.delta.io/delta-update/>
- Delta Lake Change Data Feed: <https://docs.delta.io/delta-change-data-feed/>
- Delta Lake release compatibility: <https://docs.delta.io/releases/>